

Mechanized Specifications (3): P4 and P4-SpecTec

Sukyoung Ryu

August 20, 2026

How to write a great research paper

Simon Peyton Jones
Microsoft Research Cambridge



Writing papers: model 2



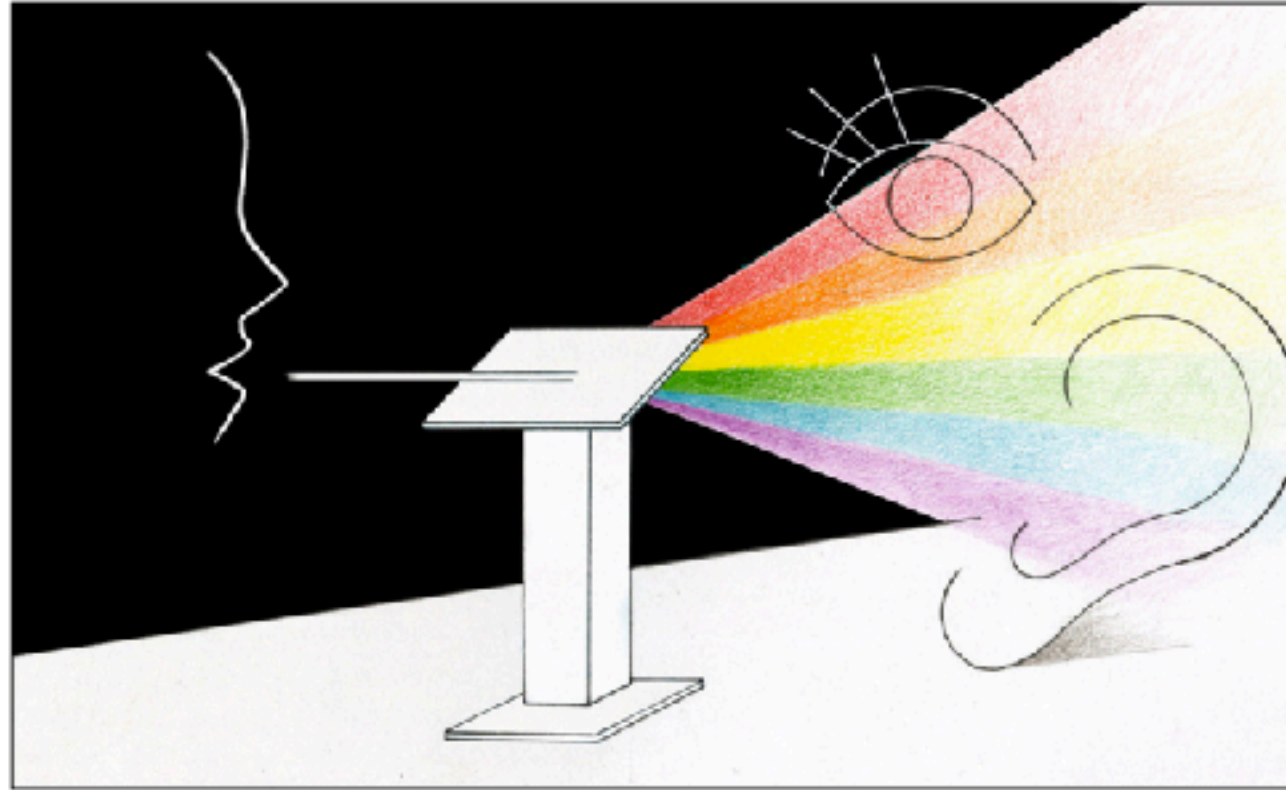
Writing papers is a primary mechanism for doing research (not just for reporting it)

Summary

1. Don't wait: write
2. Identify your key idea
3. Tell a story
4. Nail your contributions
5. Related work: later
6. Put your readers first (examples)
7. Listen to your readers

More: www.microsoft.com/research/people/simonpj

HOW TO WRITE PAPERS AND GIVE TALKS THAT PEOPLE CAN FOLLOW



Derek Dreyer, MPI-SWS
PLMW@ICFP 2022

<https://people.mpi-sws.org/~dreyer/talks/talk-plmw22icfp.pdf>



Program committee (PC) is diverse:

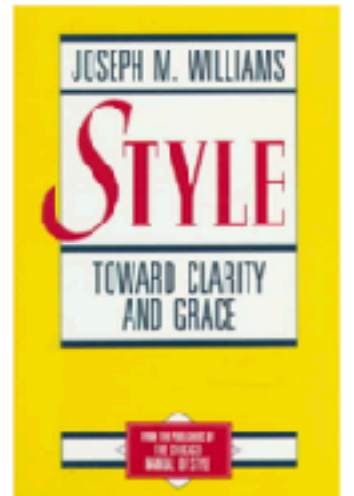
- Your reviewers may not be “experts”

Review period is short (~2 months):

- ~10-20 papers per PC member
- Often < 1 day to review each paper

Inspirations for this talk

- **Joseph M. Williams.** *Style: Toward clarity and grace.* 1990. (book)
- **Norman Ramsey.** *Learn technical writing in two hours per week.* (course notes)
 - <http://www.cs.tufts.edu/~nr/pubs/two.pdf>
- **Simon Peyton Jones.** *How to write a great research paper.* (talk)
 - <https://www.microsoft.com/en-us/research/video/how-to-write-a-great-research-paper-3/>

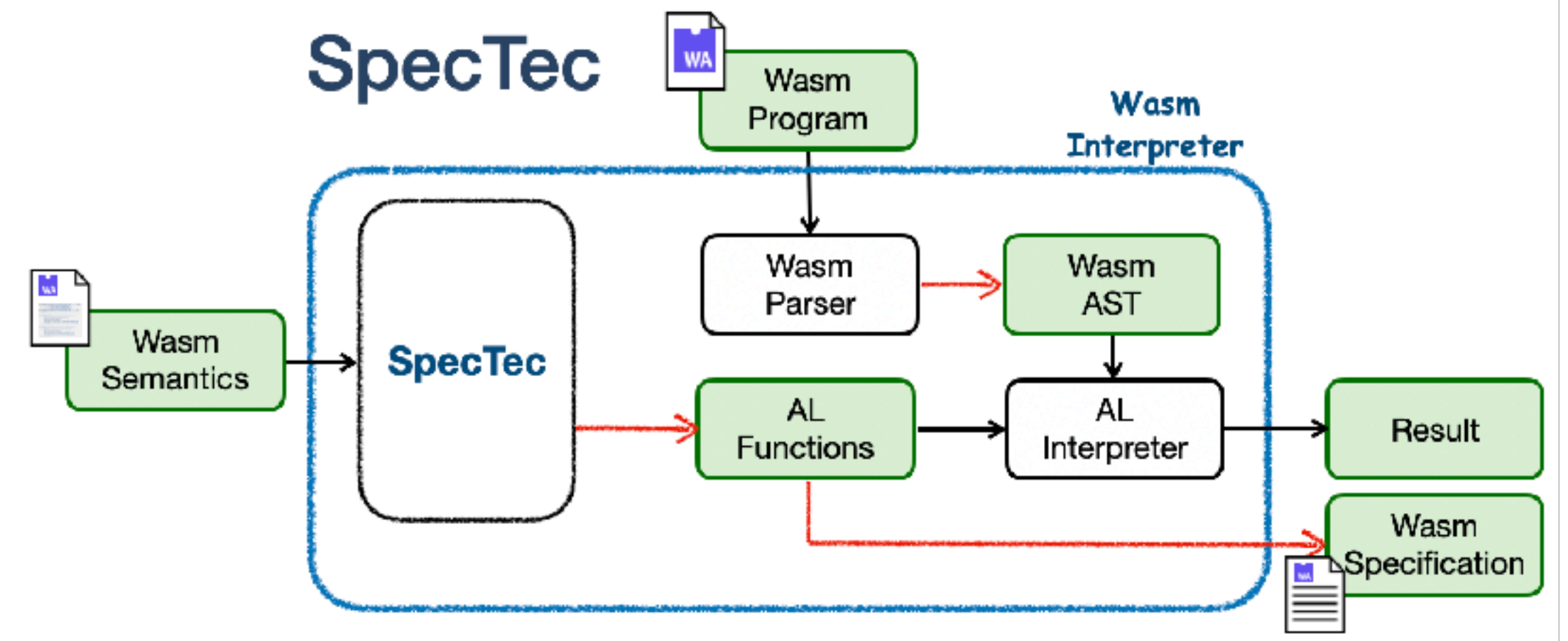


Mechanized Specifications

- **Lecture 1: JavaScript and ESMeta**
- **Lecture 2: WebAssembly and SpecTec**
- **Lecture 3: P4 and P4-SpecTec**
- **Lecture 4: Other PLs and Non-PL Software**

Wasm and SpecTec

- **March 2023** Dagstuhl seminar
- **October 2023** First presentation @ Wasm CG Meeting
- **June 2024** PLDI paper
- **June 2024** Conditional acceptance of adoption plan @ Wasm CG meeting
- **March 2025** Official acceptance of the adoption plan @ Wasm CG meeting
- **September 2025** Wasm 3.0
- **November 2025** ASE paper (ACM SIGSOFT Distinguished Paper Award)



P4-SpecTec

A Language Mechanization Framework for P4

https://www.youtube.com/watch?v=Y_1RXfUuszF0

Jaehyun Lee, Seokhun Jeong, Sukyoung Ryu
@ HATRA 2025



A DSL for the Networking Domain

A Language Mechanization Framework



Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

How do we mechanize the P4 language specification?

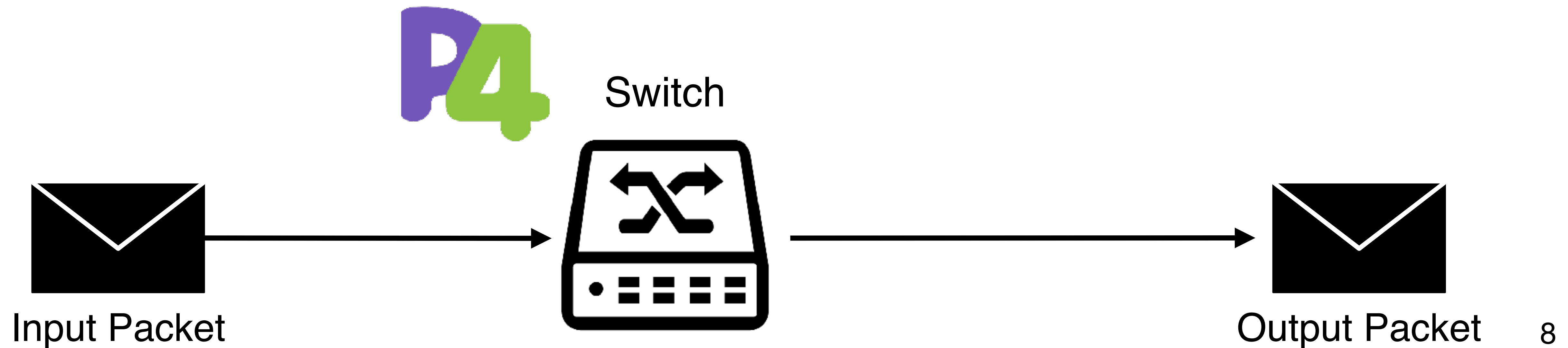
What are the benefits?

The P4 Programming Language

Programming Protocol-independent Packet Processors (P4)

A DSL to program packet-processing devices, e.g., switches

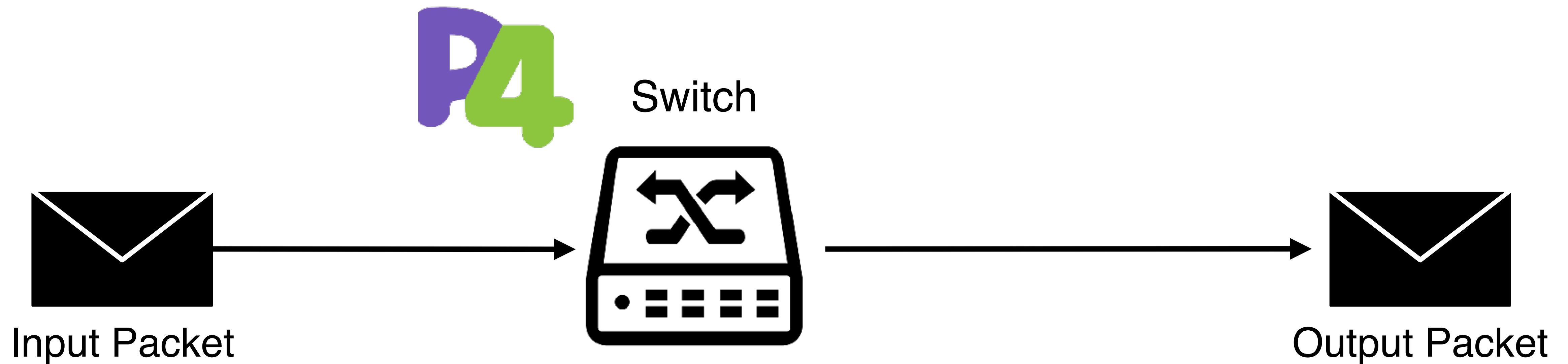
Specify how to parse & modify & emit a network packet



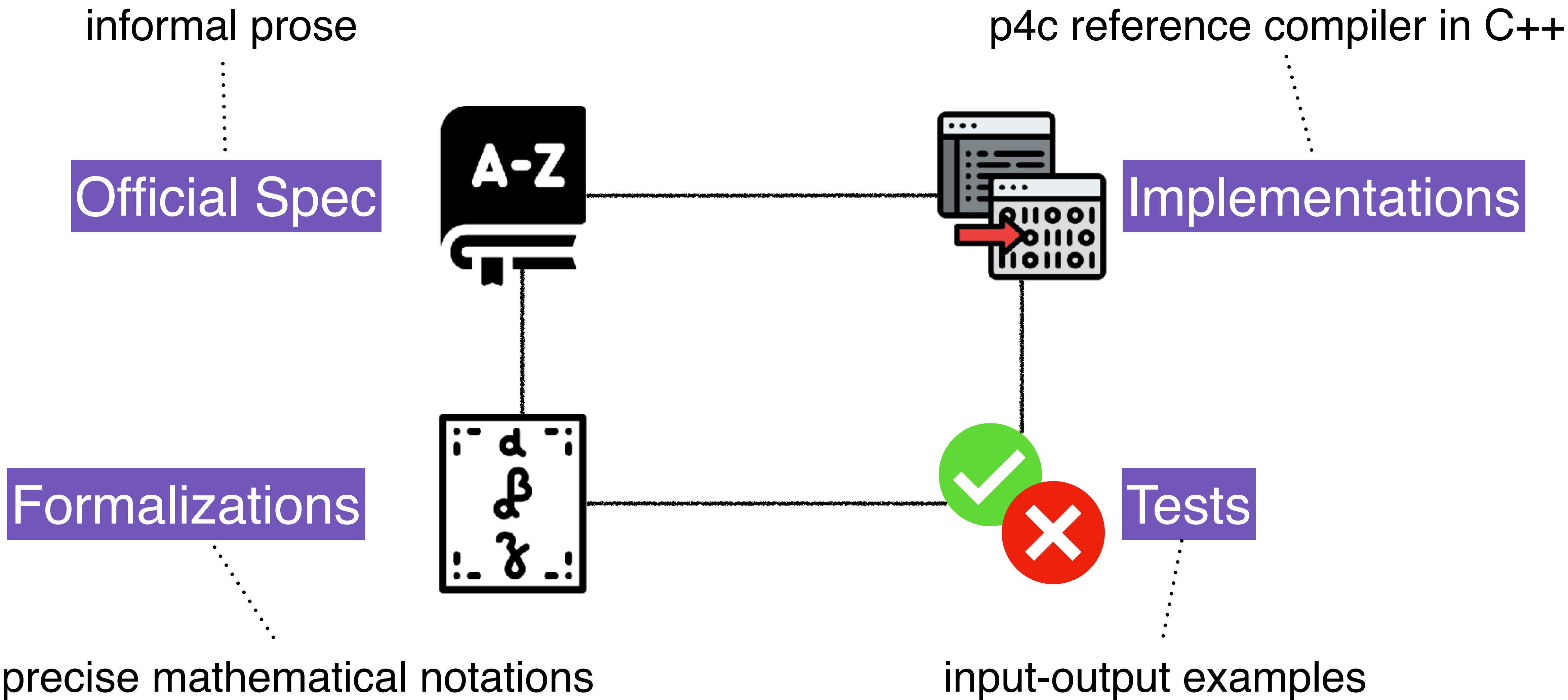
In this Talk,

Programming Protocol-independent Packet Processors (P4)

- ✓ C-like syntax and semantics
- ✓ Statically-typed

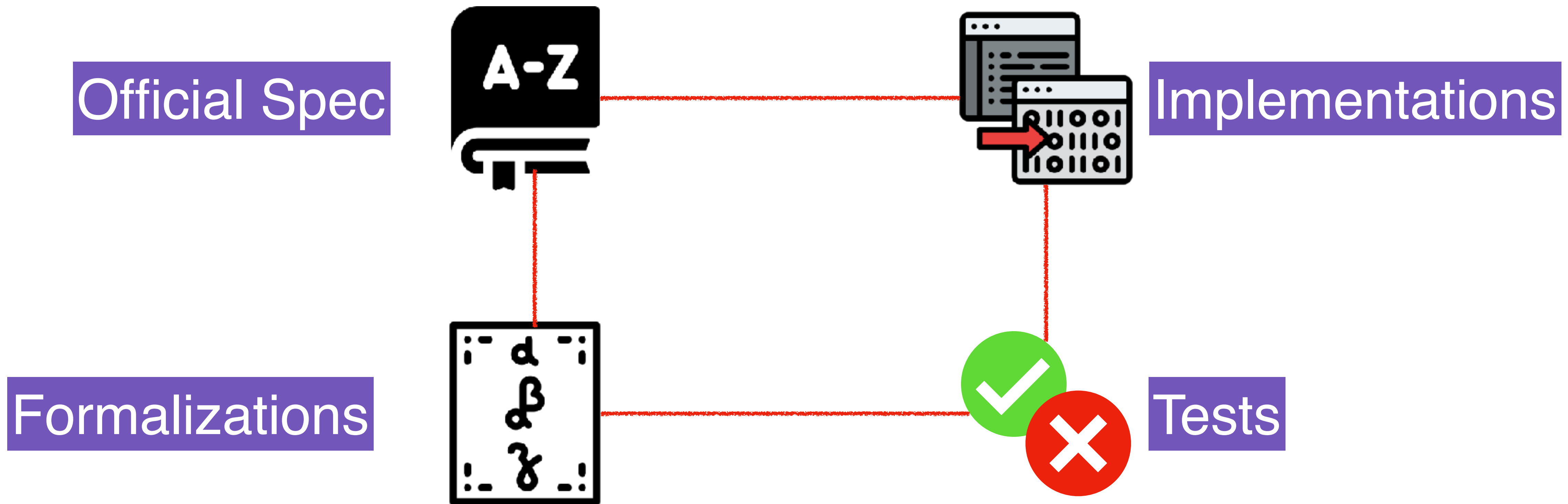


PL POV: Four representations of the P4 language



Are the four representations truly in sync?

i.e., are they **consistent** with one another? in principle, YES; in practice, NO



Maintained by different parties, and evolve at different paces

Part of the Reason: P4 is Evolving

Official Spec

Introduced generic types @ May 2021

A.3. Summary of changes made in version 1.2.2, released May 17, 2021

- Added support for accessing tuple fields (Section 8.12).
- Added support for generic structures (Section 7.2.11).

Implementations

Generics + Type Inference? Patched @ March 2025

Compiler Bug: Could not find type of <Type_Header> ... on specialized generic struct type #4835

✓ Closed

Bug

#5133

Formalizations

Remains pre-2021, the recent version is based 2024

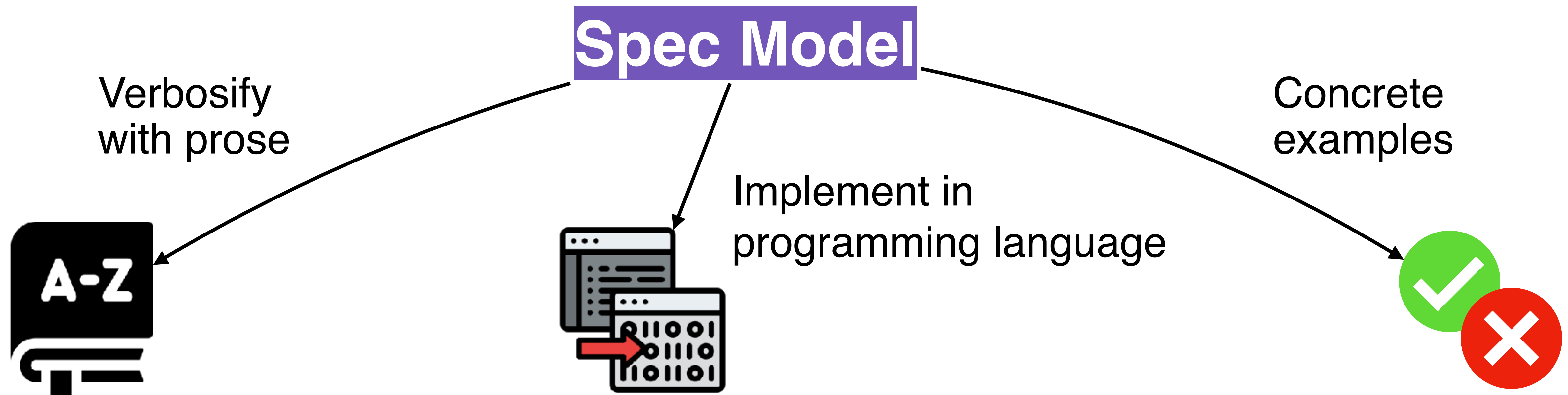
Can we, at the same time,
evolve a programming language,
and maintain the consistency between representations?

Single Source of Truth Underlying the Representations

The **Specification Model**,

an unambiguous and complete definition of the syntax and semantics.

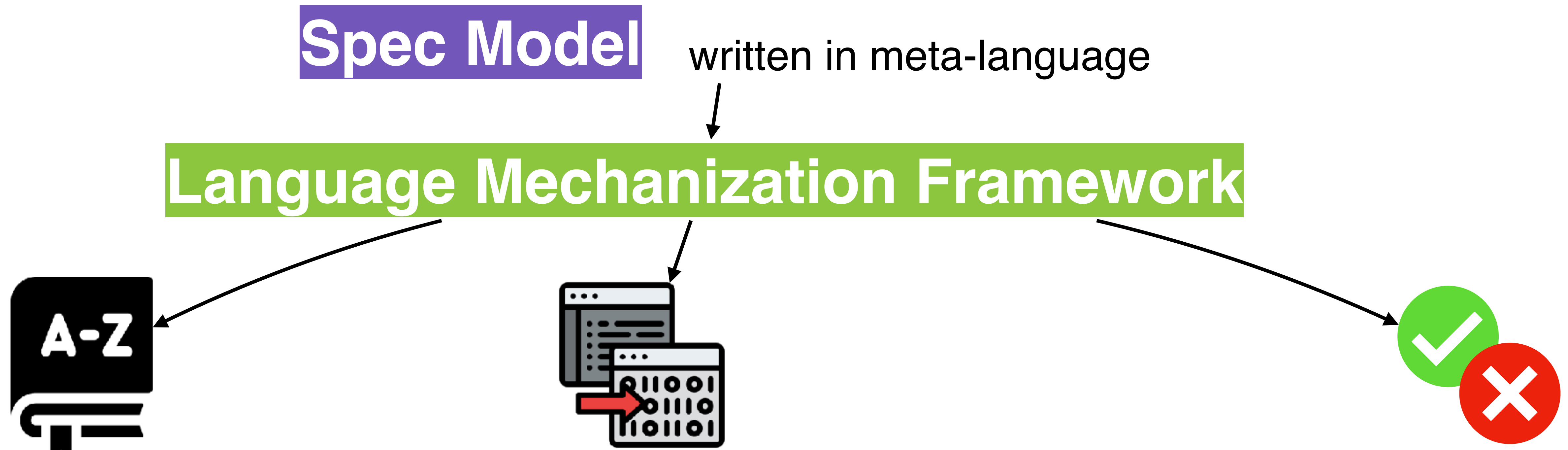
e.g., complete formalization, definitional interpreter



Language Mechanization Frameworks for Consistency

Language mechanization frameworks

provide a DSL (called meta-language) for defining the spec model
and automate generation of various backends from it





Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

How do we mechanize the P4 language specification?

What are the benefits?

Mechanization Integrated in Real-World Languages

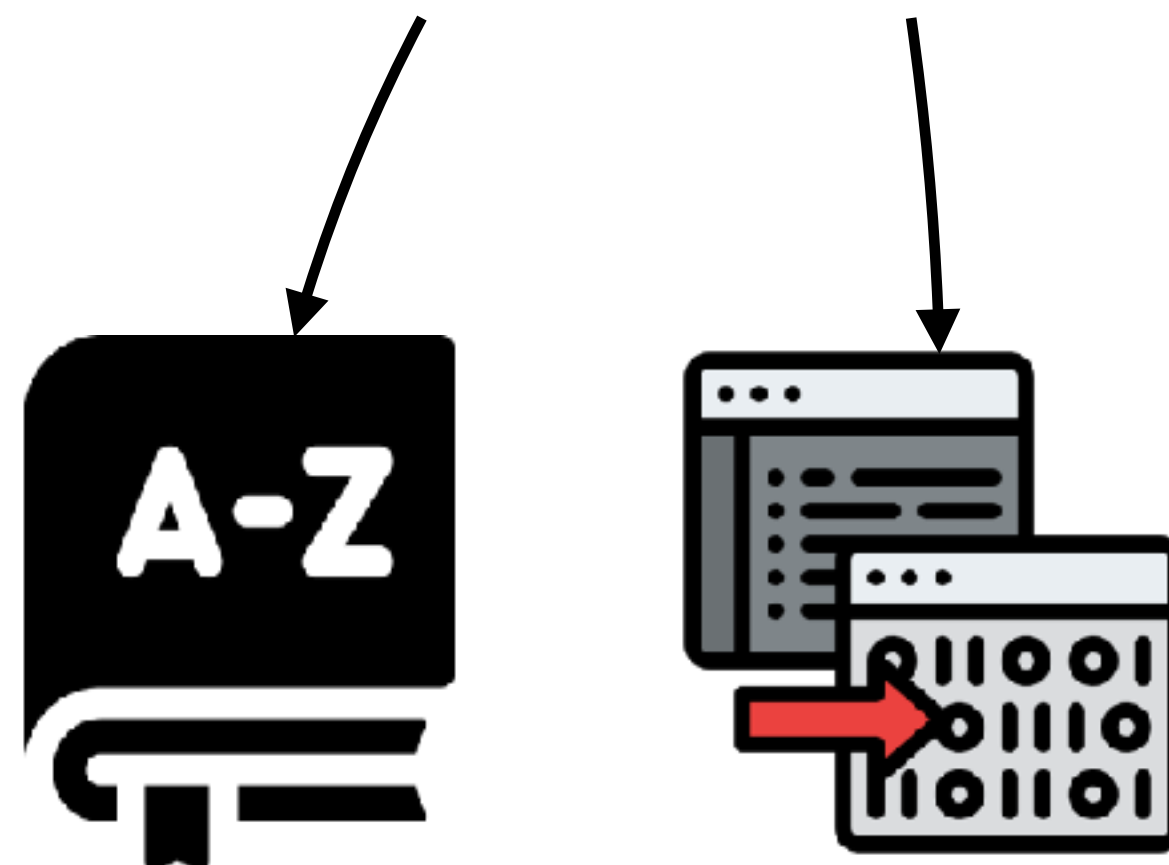


Wasm-SpecTec

SpecTec has been adopted

Published on March 27, 2025 by [Andreas Rossberg](#) .

<https://webassembly.org/news/2025-03-27-spectec/>



Underlying Recipe: An Authoritative Spec Model

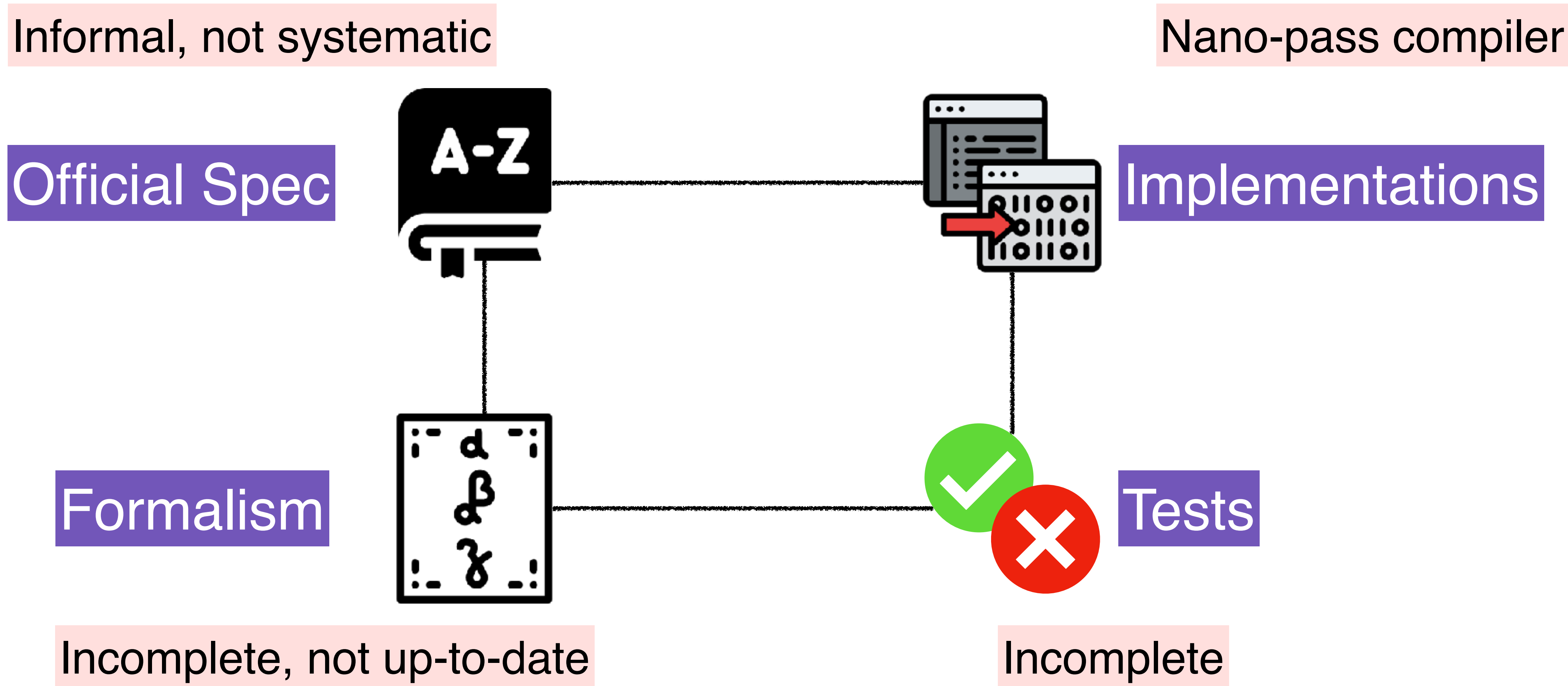
THE standard

```
(i32.const c) if bt instr1* else instr2* end ↪ block bt instr1* end  
                                                    (if c ≠ 0)  
(i32.const c) if bt instr1* else instr2* end ↪ block bt instr2* end  
                                                    (if c = 0)
```

drives the spec model design

Formal Reduction Rules

P4 Lacks an Authoritative Spec Model



Can we reuse Wasm-SpecTec meta-language for P4?

Declarative Inference (Reduction) Rules

Hide implementation-level details

Enforce case-analysis by *disallowing* nested control flow



```
LHS ~> RHS
-- if cond:
  -- ... foo ...
-- else:
  -- ... bar ...
```



```
LHS ~> RHS
-- if cond:
  -- ... foo ...
```



```
LHS ~> RHS
-- if not cond:
  -- ... bar ...
```

Can we reuse Wasm-SpecTec meta-language for P4?

Declarative Inference Rules

Inference rules are not directly executable

Can handle Wasm dynamic semantics but,

```
rule Type_sub/trans:  
  C |- type_a <: type_c  
  -- Type_sub: C |- type_a <: type_b  
  -- Type_sub: C |- type_b <: type_c
```

cannot handle static semantics, *cannot* generate a type checker

For P4: Algorithmic Inference Rules

Declarative Inference Rules

Abstract, yet not directly executable

Algorithmic Inference Rules

Subset of inference rules that are constructive

Wasm-SpecTec meta-language with constraints

P4 Spec Model

written in meta-language



P4-SpecTec

Constraint 1. Annotate relation input/outputs

Constraint 2. Data flows through inputs, premises, outputs

Constraint 3. All variables must be known except in binders

Constraint 4. ``=`` is disambiguated into ``==``, ``:=``, or ``=:``

... and more ...

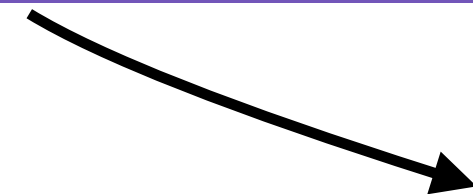
The P4 Spec Model

Full P4 spec model of syntax and type system (excluding P4 annotations)

~ 8K LoC in meta-language

P4 Spec Model

written in meta-language



P4-SpecTec



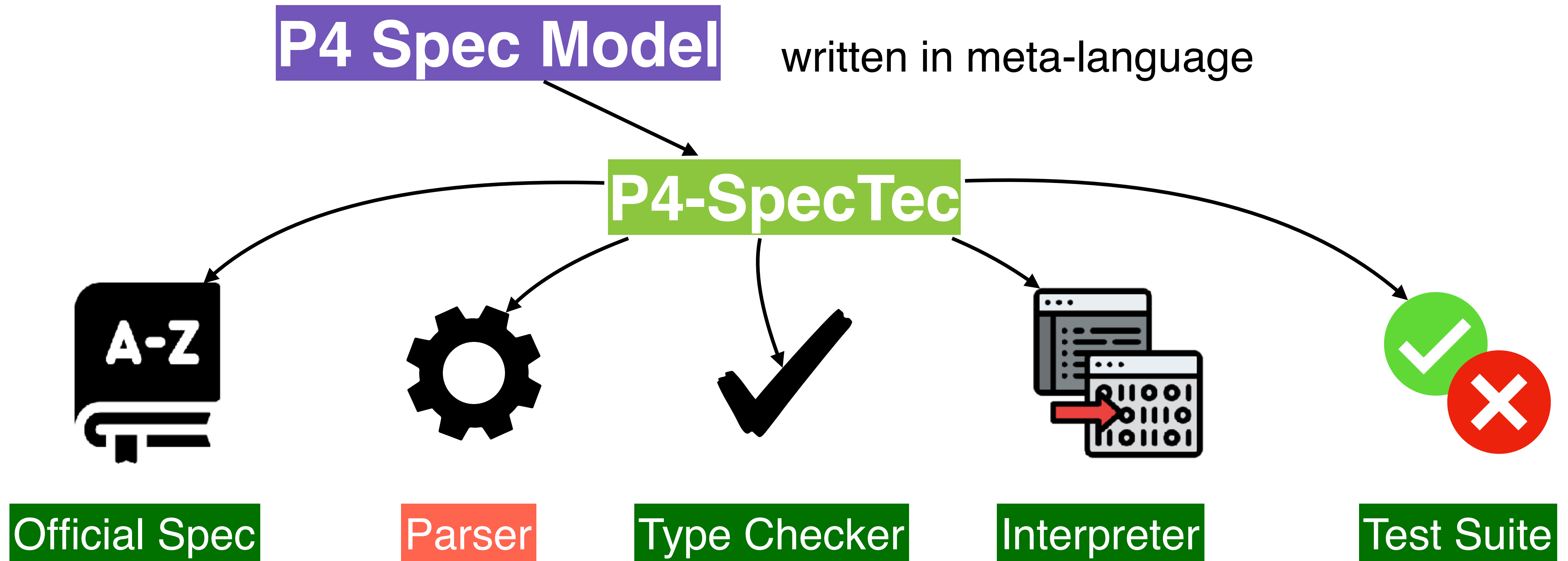
Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

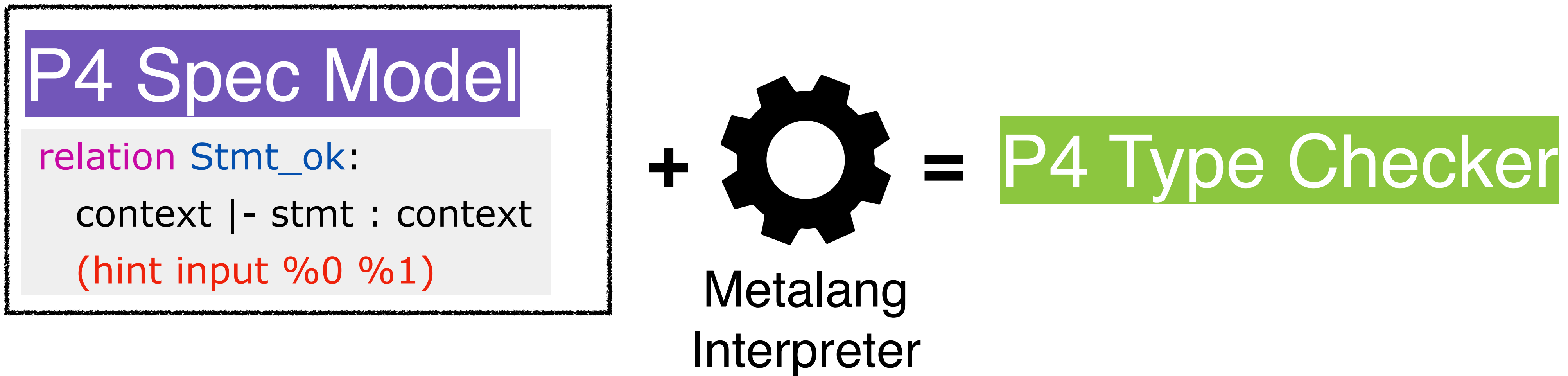
How do we mechanize the P4 language specification?

What are the benefits?

P4-SpecTec Vision and Current Status



P4-SpecTec Type Checker Backend



Validated with the p4c reference compiler test suite:

- Positive tests: 1063/1067 (99.6%)
- Negative tests: 254/258 (98.4%)

P4-SpecTec Negative Type Checker Test Backend

Added negative type tests for previously untested type errors

Add 248 negative tests to `p4_16_errors` #5369

 Merged fruffy merged 2 commits into `p4lang:main` from `KunJeong:main`  last month

Discovered 26 previously unknown bugs in the p4c compiler frontend

Fails to reject bitwise operation on arbitrary precision integers #5300

 Open

Bug

Failing with Purpose: Dangling Coverage-Guided Negative Test Generation from a Mechanized P4 Type System

<https://dl.acm.org/doi/10.1145/3797109>

Jaehyun Lee, Seokhun Jeong, Sukyoung Ryu
@ FSE 2026

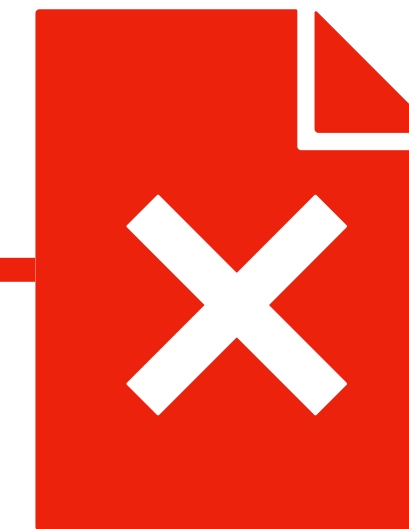


A Correct P4 Type Checker would:

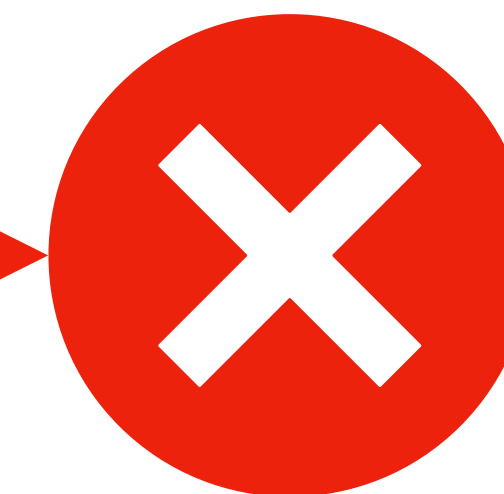
Well-typed program



Ill-typed program



P4 Type Checker



Testing the P4 Type Checker

Positive tests

Well-typed program

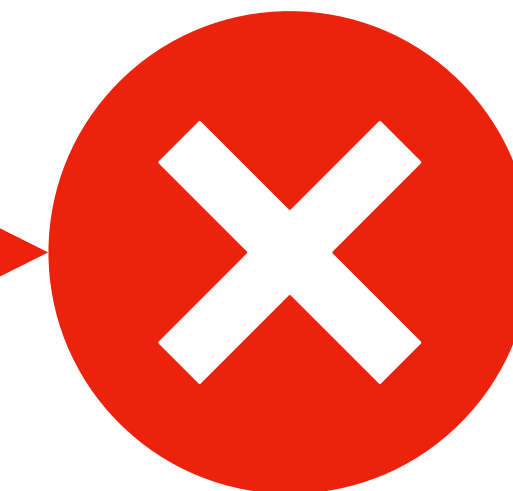


Negative tests

Ill-typed program



P4 Type Checker



The P4 Type System, according to the Official P4 Spec

8.10.2. A note about shifts

The left operand of shifts can be any one out of unsigned bit-strings, signed bit-strings, and arbitrary-precision integers, and the right operand of shifts must be either an expression of type `bit<S>` or a compile-time known value that is a non-negative integer. The result has the same type as the left operand.

The RHS of a shift should be:

- of type `bit<S>` or,
- if `int<S>`, then should be a compile-time known non-negative value

A Well-Typed Shift Operation

```
bit<8> shl(in bit<8> x,  
           in bit<8> y) {  
    return x << y;  
}
```



```
bit<8> shl(in bit<8> x) {  
    const int<8> y = 42;  
    return x << y;  
}
```



The RHS of a shift should be:

- of type `bit<S>` or,
- if `int<S>`, then should be a compile-time known non-negative value

An Ill-Typed Shift Operation

```
bit<8> shl(in bit<8> x, in int<8> y) {  
    return x << y;  
}
```



The RHS of a shift should be:

- of type `bit<S>` or,
- if `int<S>`, then should be a compile-time known non-negative value

Existing Negative Tests are Insufficient

Of 254 official negative tests,
NONE targets the condition on shift RHS



The RHS of a shift should be:

- of type `bit<S>` or,
- if `int<S>`, then should be a compile-time known non-negative value

Existing Negative Tests are Insufficient

Of 254 official negative tests,
NONE targets the condition on shift RHS



Shift by a string literal is not rejected by type checker #5094

✓ Closed

Bug

#5446

Compiler Bug

Goal: A Comprehensive Negative Test Suite

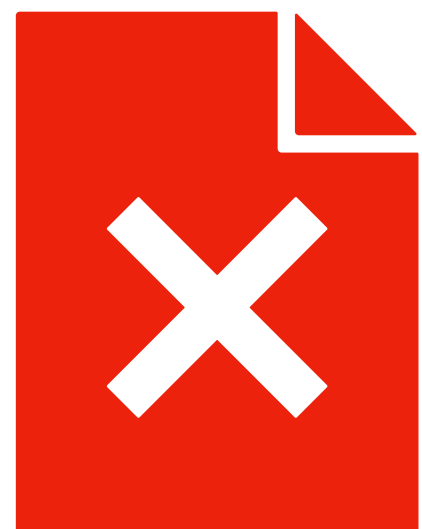
⋮

At least **one test for each ill-typed condition** in the P4 type system



8.10.2. A note about shifts

The left operand of shifts can be any one out of unsigned bit-strings, signed bit-strings, and arbitrary-precision integers, and the right operand of shifts must be either an expression of type `bit<S>` or a compile-time known value that is a non-negative integer. The result has the same type as the left operand.



```
bit<8> shl(in bit<8> x, in int<8> y) {  
    return x << y;  
}
```

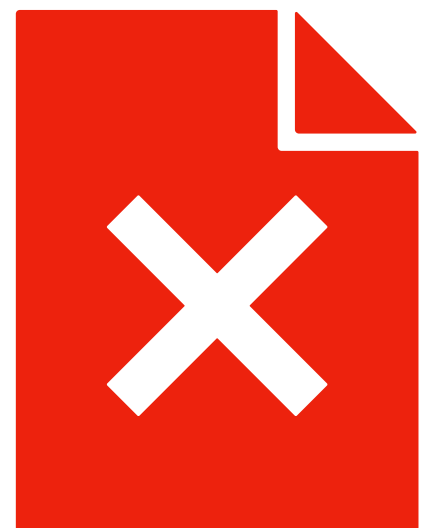
Challenge: Quantifying the Comprehensive-ness



P4 spec in *natural language*:
ambiguous and incomplete



No systematic characterization of
the P4 ill-typed conditions

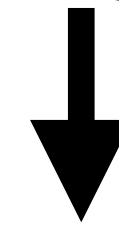


Difficult to correlate negative tests
to the ill-typed conditions

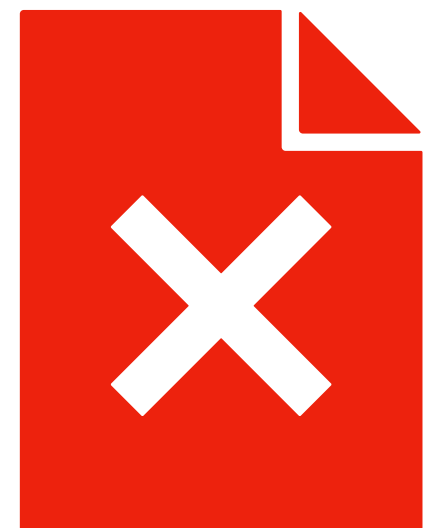
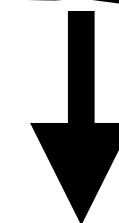
Our Work



Define a *mechanized*
formal P4 type system *specification*



Define *dangling coverage* of a spec,
quantifying the ill-typed conditions



Generate negative tests,
achieving high dangling coverage

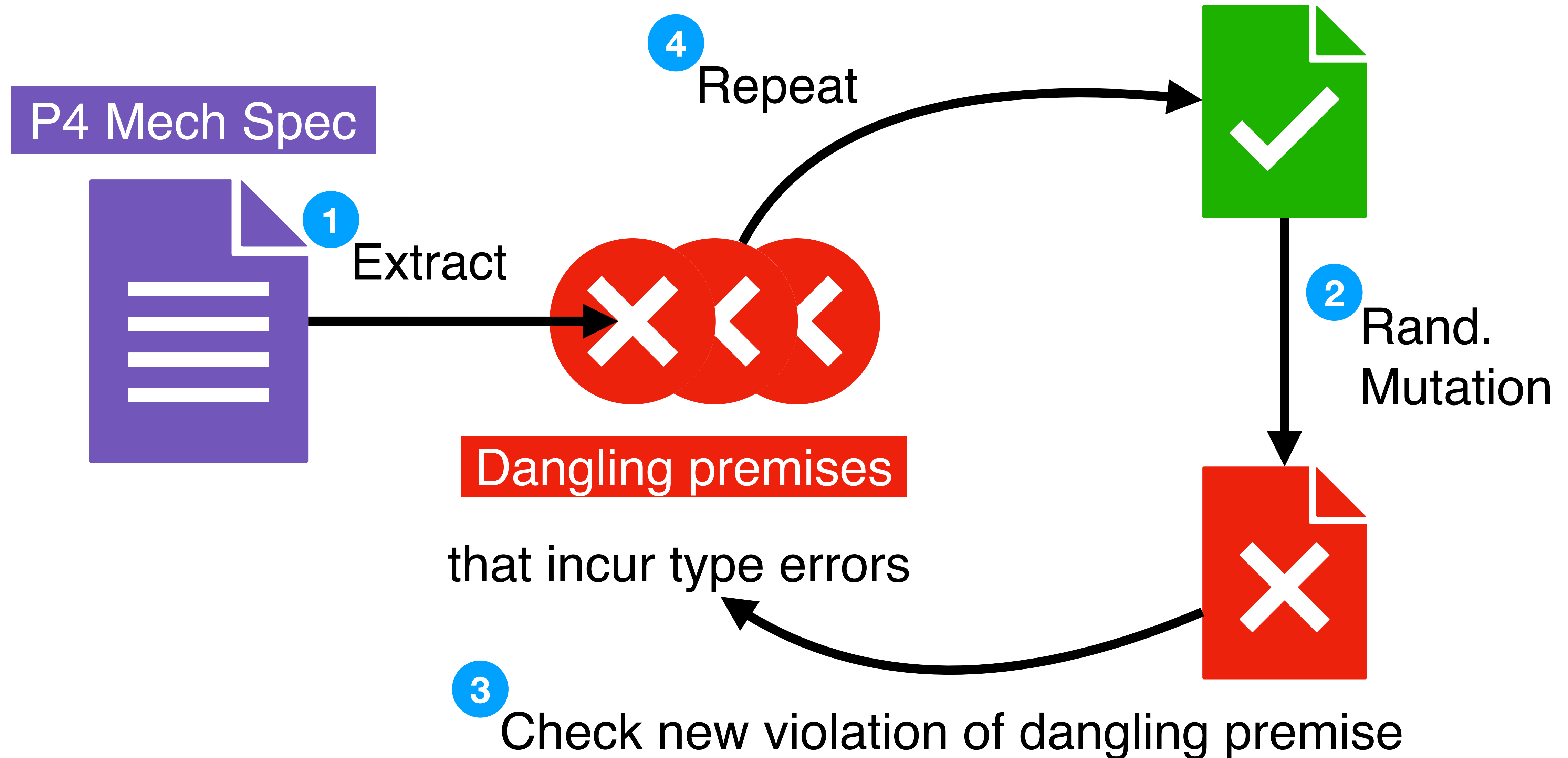
A Mechanized P4 Type System Spec

P4 Mech Spec



1. Specified with **SpecTec*** spec language
2. A **formal description** of P4 syntax and semantics
3. Can parse the spec, treating it as data
4. Can **execute** the spec
 - Run the P4 type system spec = P4 type checker

Negative Test Generation from a *Mechanized Spec*



Formal Type System, as a Mechanized Spec

The RHS of a shift should be:

- of type `bit<S>` or,
- if `int<S>`, then should be a compile-time known non-negative value

P4 Mech Spec



```
syntax binop =  
  | PLUS | MINUS  
  | SHL | SHR
```

```
syntax type = IntT | BitT
```

```
syntax expr =  
  | VarE text  
  | IntE int  
  | BitE nat  
  | BinE binop expr expr
```

Type System Spec Using *Inference Rules*

For C and BinE ..., if all premises hold, then the result of type checking is type_l .

rule $\text{Expr_ok}/\text{binary-shift}$:

$C \vdash \text{BinE binop expr_l expr_r} : \text{type_l}$

-- if $\text{binop} \in \{ \text{SHL}, \text{SHR} \}$

-- $\text{Expr_ok}: C \vdash \text{expr_l} : \text{type_l}$

-- $\text{Expr_ok}: C \vdash \text{expr_r} : \text{type_r}$

-- if $\text{\$is_int}(\text{type_r}) \Rightarrow \text{\$is_non_negative}(C, \text{expr_r})$

Executing the Inference Rule(s)

```
rule Expr_ok/binary-shift:
```

```
  C ⊢ BinE binop expr_l expr_r : type_l
```

```
----
```

```
-- if binop ∈ { SHL, SHR }
```

```
-- Expr_ok: C ⊢ expr_l : type_l
```

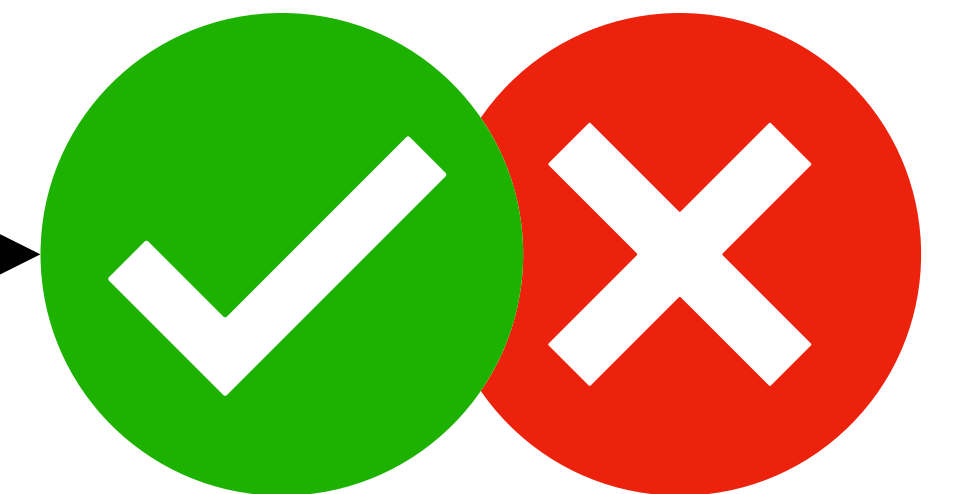
```
-- Expr_ok: C ⊢ expr_r : type_r
```

```
-- if $is_int(type_r) => $is_non_negative(C, expr_r)
```

P4 Program



Spec Interpreter



Well-Typed P4 Program

rule Expr_ok/binary-shift:

C ⊢ BinE binop expr_l expr_r : type_l

-- if binop ∈ { SHL, SHR }

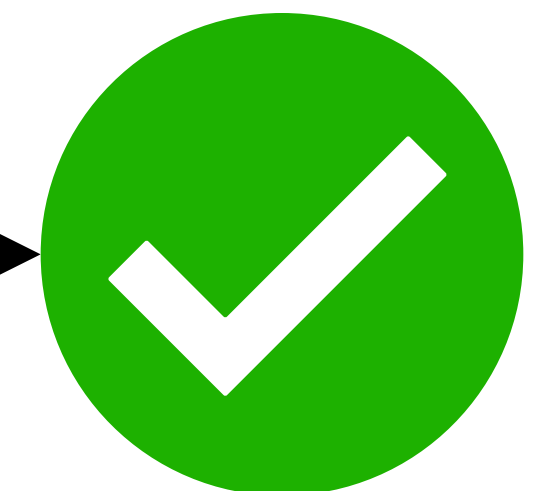
-- Expr_ok: C ⊢ expr_l : type_l

-- Expr_ok: C ⊢ expr_r : type_r

-- if \$is_int(type_r) => \$is_non_negative(C, expr_r)

```
bit<8> shl(in bit<8> x,  
           in bit<8> y) {  
    return x << y;  
}
```

Spec Interpreter



Ill-Typed P4 Program

rule Expr_ok/binary-shift:

C ⊢ BinE binop expr_l expr_r : type_l

-- if binop ∈ { SHL, SHR }

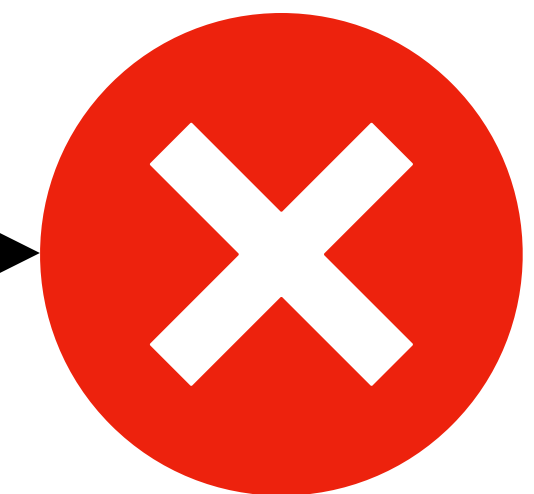
-- Expr_ok: C ⊢ expr_l : type_l

-- Expr_ok: C ⊢ expr_r : type_r

-- if \$is_int(type_r) => \$is_non_negative(C, expr_r)

```
bit<8> shl(in bit<8> x,  
           in int<8> y) {  
    return x << y;  
}
```

Spec Interpreter



Inference Rules **ONLY** define **well-typed conditions**

If **violated**, corresponds to an **ill-typed condition**

```
rule Expr_ok/binary-shift:
```

```
  C ⊢ BinE binop expr_l expr_r : type_l
```

```
----
```

```
-- if binop ∈ { SHL, SHR }
```

```
-- Expr_ok: C ⊢ expr_l : type_l
```

```
-- Expr_ok: C ⊢ expr_r : type_r
```

```
-- if $is_int(type_r) => $is_non_negative(C, expr_r)
```

Yet, Not ALL Premises are Ill-typed Conditions

rule Expr_ok/binary-shift:

$C \vdash \text{BinE binop expr}_l \text{ expr}_r : \text{type}_l$

-- if binop $\in \{ \text{SHL}, \text{SHR} \}$

rule Expr_ok/binary-plus-minus:

$C \vdash \text{BinE binop expr}_l \text{ expr}_r : \text{type}_l$

-- if binop $\in \{ \text{PLUS}, \text{MINUS} \}$

When violated,
applies another rule;
Performing case-analysis

Counterpart premise

Goal: Filter-out Premises for Case-analysis

rule Expr_ok/binary-shift:

$C \vdash \text{BinE binop expr}_l \text{ expr}_r : \text{type}_l$

-- if $\text{binop} \in \{ \text{SHL}, \text{SHR} \}$

-- Expr_ok: $C \vdash \text{expr}_l : \text{type}_l$

-- Expr_ok: $C \vdash \text{expr}_r : \text{type}_r$

-- if $\text{is_int}(\text{type}_r) \Rightarrow \text{is_non_negative}(C, \text{expr}_r)$

A counterpart premise exists
NOT a type error when violated

No counterpart premise exists
A type error when violated (i.e., an ill-typed condition)

Finding *Dangling Premises*

P4 Mech Spec



Premise *with* a counterpart

```
-- if binop ∈ { SHL, SHR }
```

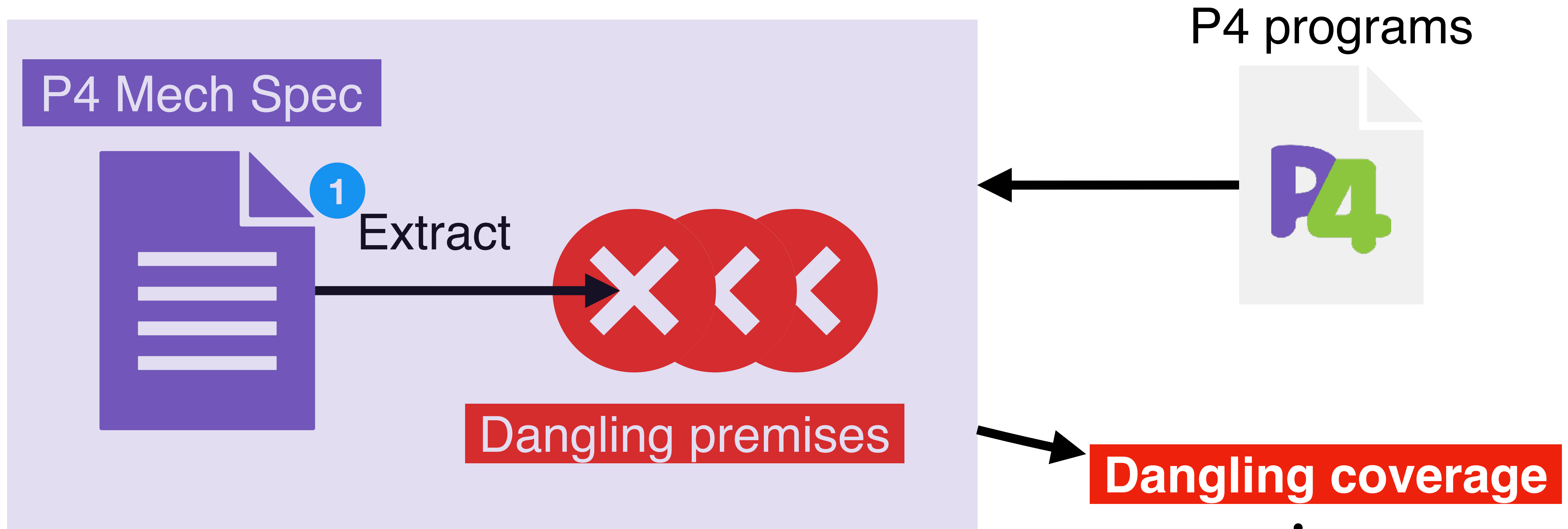
```
-- if binop ∈ { PLUS, MINUS }
```

Dangling premise

```
-- if $is_int(type_r) => $is_non_negative(C, expr_r)
```

... details in our paper :)

Measuring *Dangling Coverage*



Quantifies the comprehensive-ness
of a negative test suite

Type Errors are One-off from a Well-typed Program

```
bit<8> shl(in bit<8> x,  
           in bit<8> y) {  
    return x << y;  
}
```



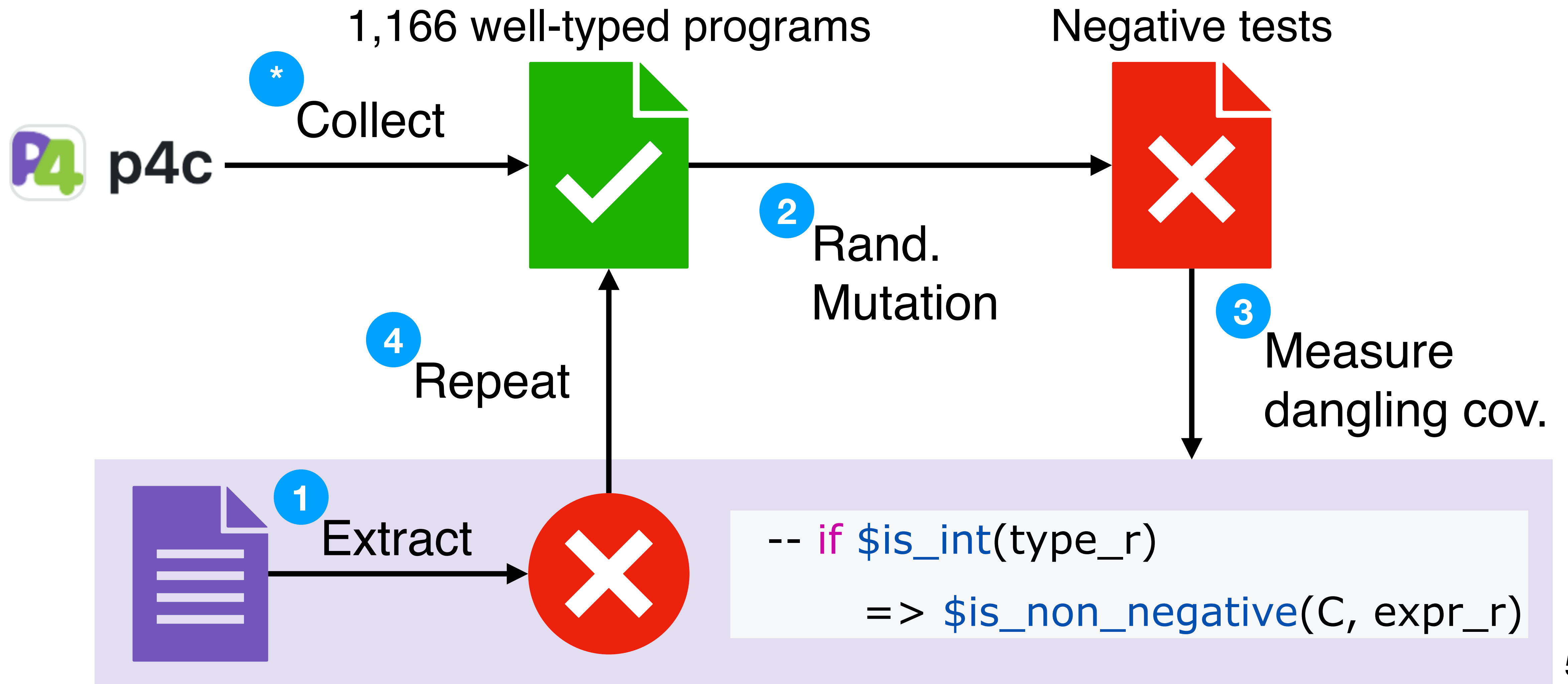
```
bit<8> shl(in bit<8> x,  
           in int<8> y) {  
    return x << y;  
}
```



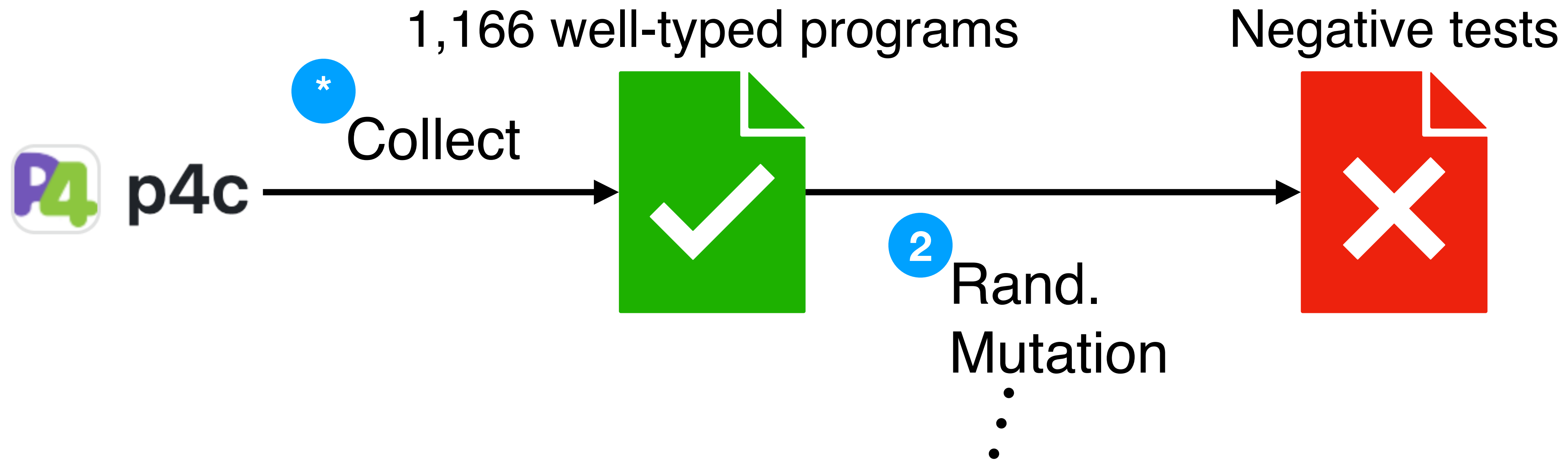
Violation of a dangling premise

```
-- if $is_int(type_r)  
    => $is_non_negative(C, expr_r)
```

Dangling Coverage-Guided Negative Test Generation



What to mutate?



1. Which well-typed program to mutate?
 2. Which part of the well-typed program to mutate?
- ... details in our paper :)

Evaluation

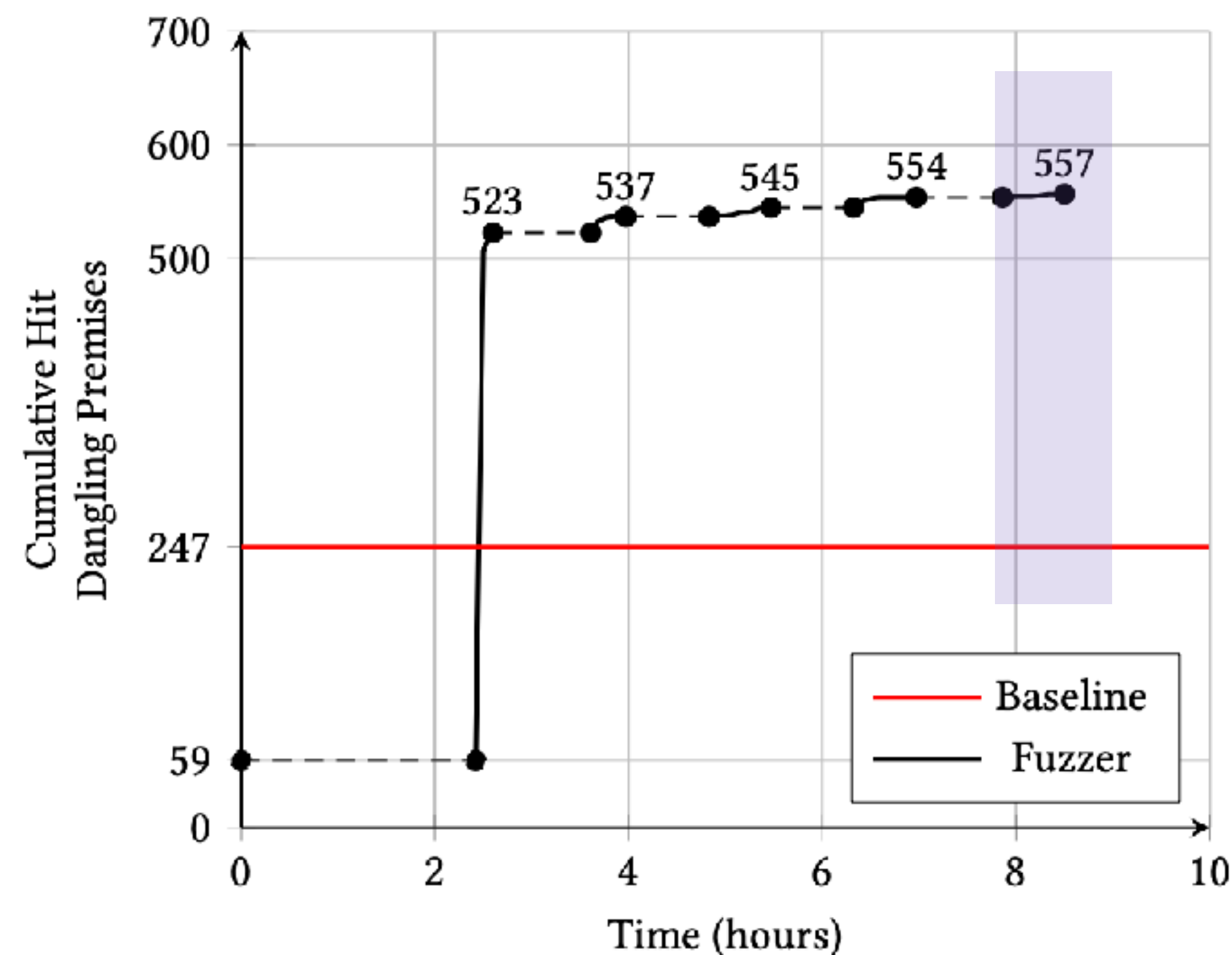
1. Does it achieve **higher dangling coverage** than the official test suite?
2. Effectiveness of mutation strategies?
3. **Bug finding ability** of the fuzzer?
4. Sensitivity of dangling coverage w.r.t the negative input space?
5. Comparison with existing P4 fuzzers?

Higher dangling coverage than the official test suite?

Extracted 939 dangling premises from the mechanized spec

Ran 10 hours of fuzzing

using 1,166 well-typed programs from the official test suite



- Generated 422 negative tests
 - Baseline (Red): 247/939 (26.30%)
 - Generated (Black): 557/939 (59.32%)
- +33.02%p**

Can the generated tests identify compiler bugs?

Generated
negative tests



Official P4 Compiler's Type Checker



15 Soundness bugs



14 Crash bugs

Can the generated tests identify compiler bugs?

Generated
negative tests



Official P4 Compiler's Type Checker



Add 248 negative tests to `p4_16_errors` #5369

Merged

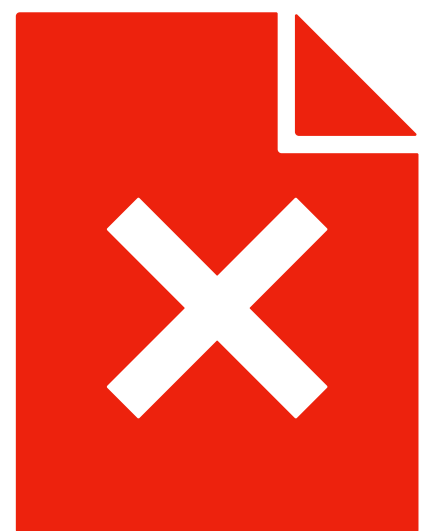
`fruffy` merged 2 commits into `p4lang:main` from `KunJeong:main` on Sep 12, 2025



Define a *mechanized*
formal P4 type system *specification*



Define *dangling coverage* of a spec,
quantifying the ill-typed conditions



Generate negative tests,
guided by dangling coverage

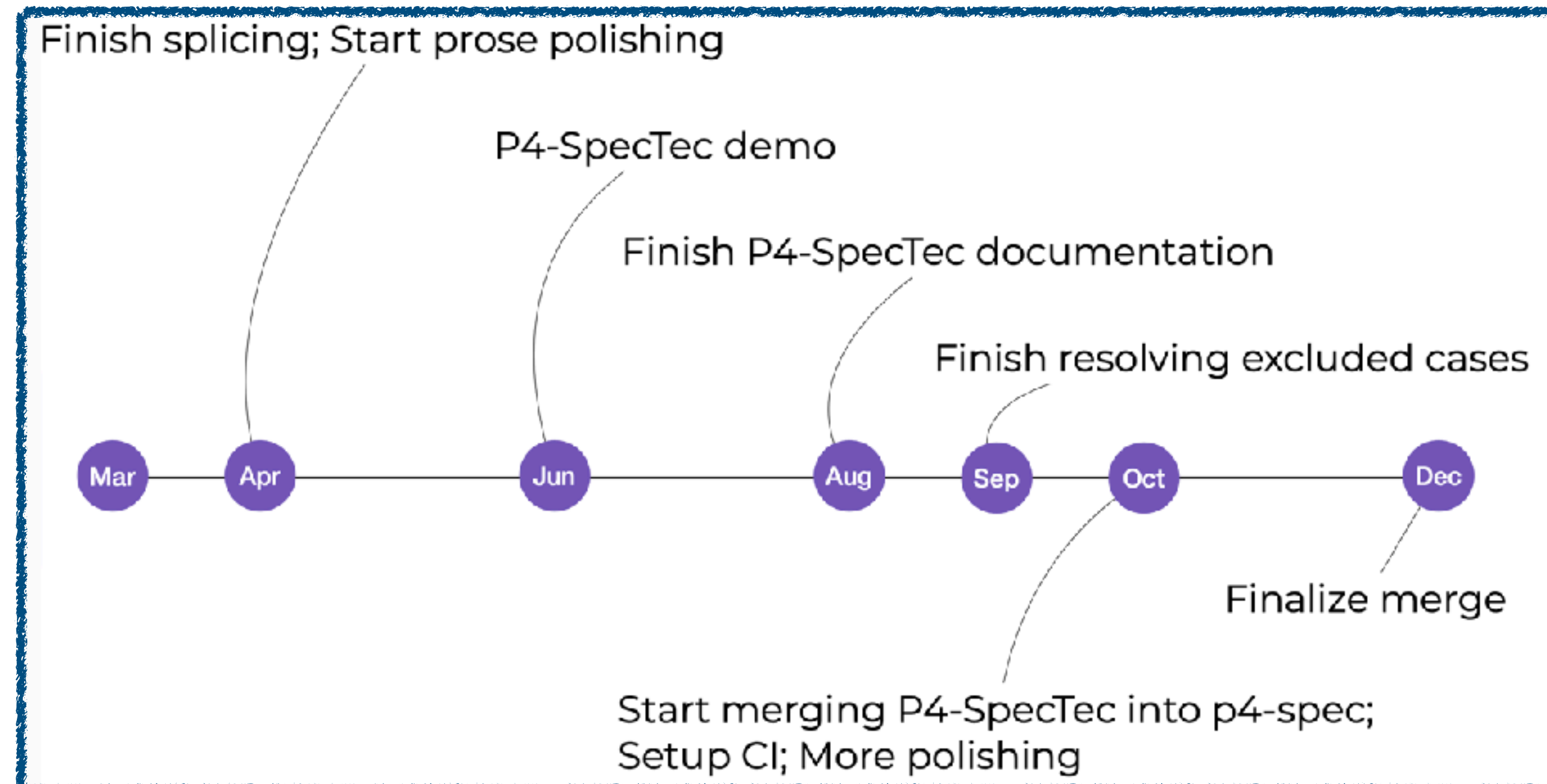
Adopting P4-SpecTec as the official language specification authoring toolchain

2 Mar 2026

[Mailing list discussion](#)

<https://bit.ly/4gnMMJy>

- Presentation ([slides](#))
- **The P4 Language Design Working Group has decided to officially conditionally accept P4-SpecTec as the official language specification authoring toolchain for P4.**
 - We hope to transition this to an unconditional acceptance by the end of 2026, given that the timeline in slide 13 above is followed and the questions asked by the community below are handled satisfactorily.



P4-SpecTec: Integrating a Language Mechanization Framework into the Real-World P4 Specification

JAEHYUN LEE✉, KAIST, Republic of Korea
SEOKHUN JEONG, KAIST, Republic of Korea
SEHYUK AHN, KAIST, Republic of Korea
HAECHAN KWON, KAIST, Republic of Korea
SUKYOUNG RYU, KAIST, Republic of Korea

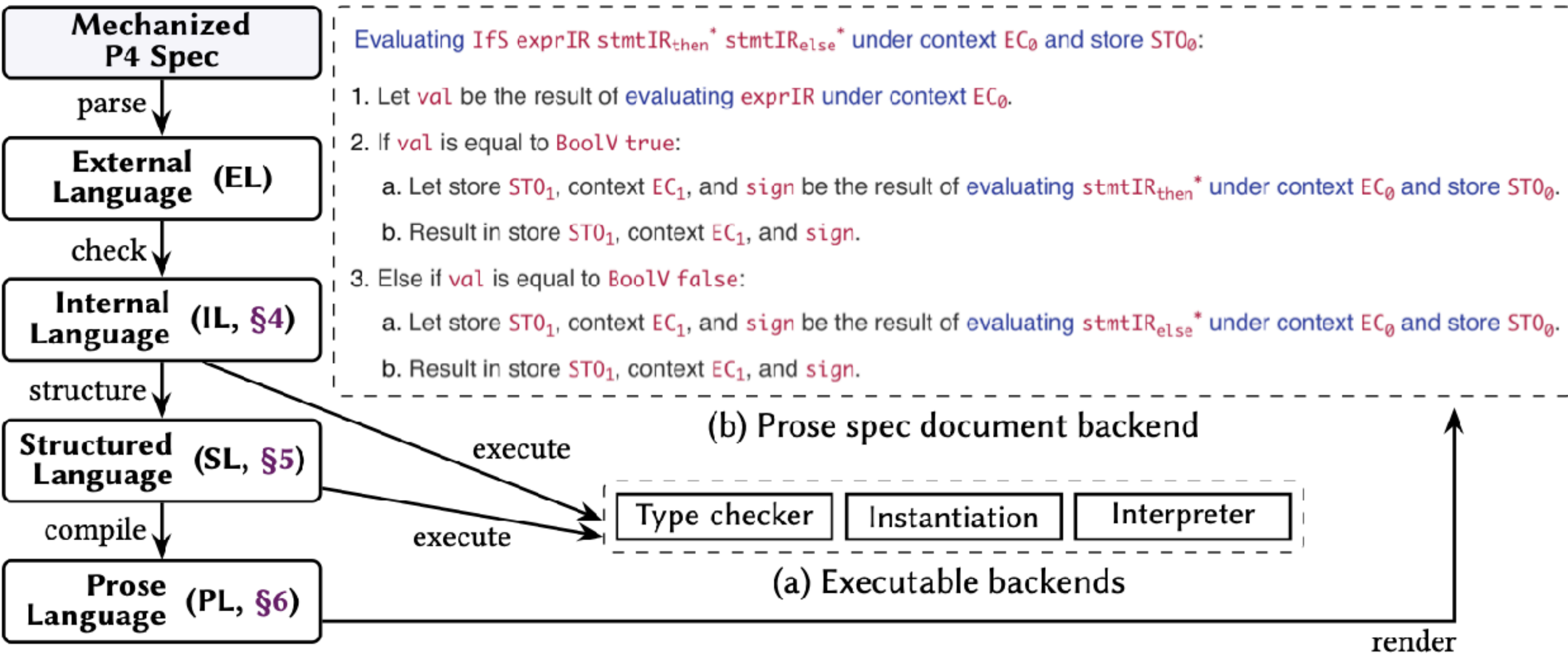
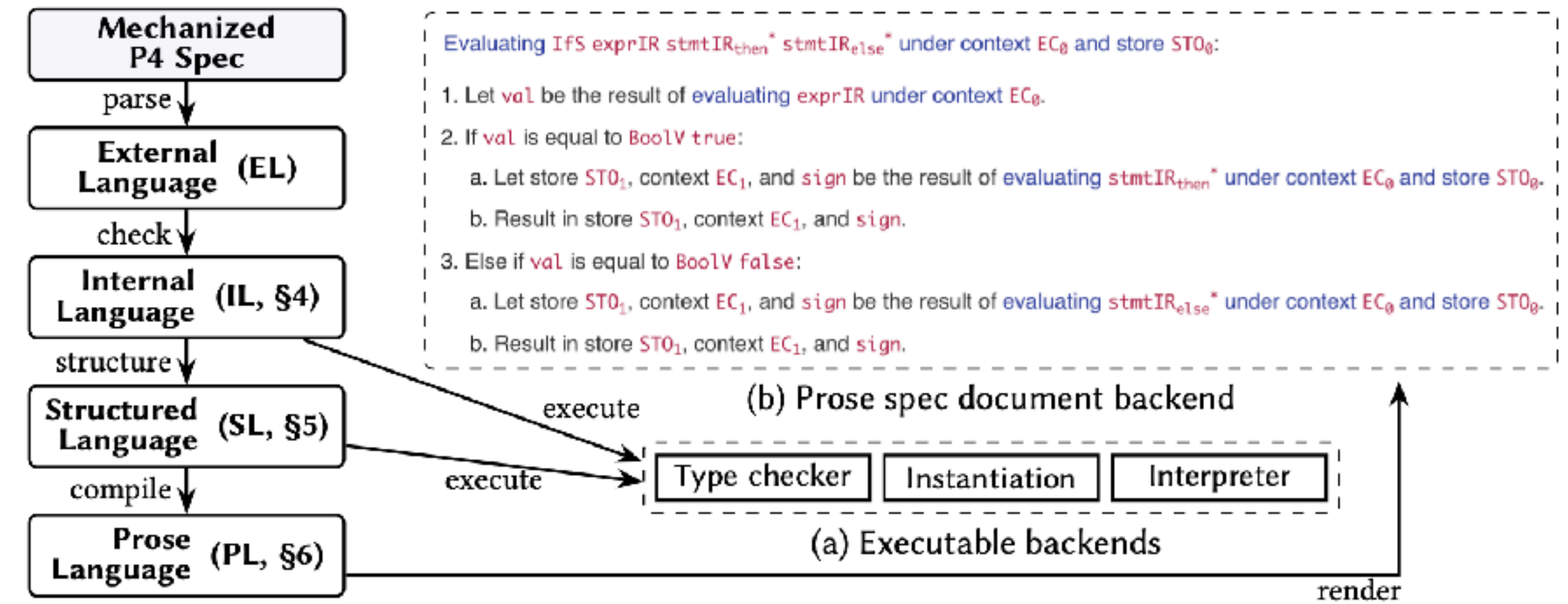


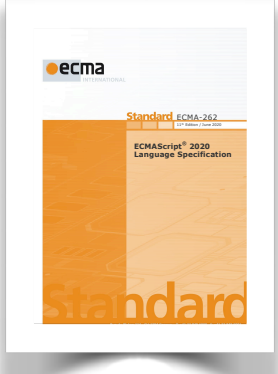
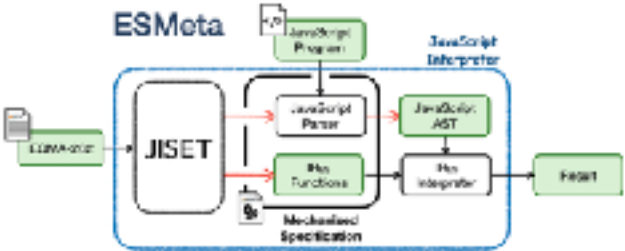
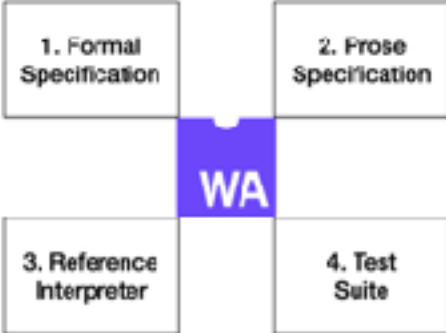
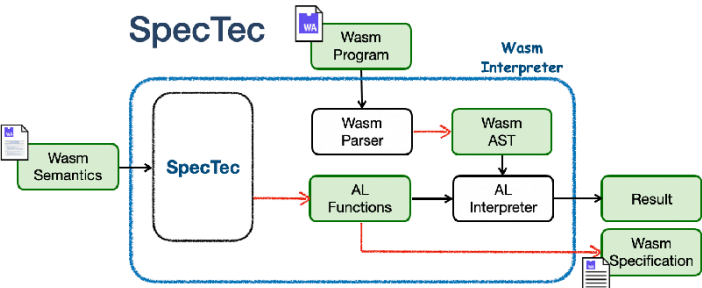
Fig. 10. Overview of the P4-SpecTec architecture.

P4 and P4-SpecTec

- **January 2024** First meeting
- **October 2024** Presentation @ P4 Workshop
- **December 2024** First presentation @ P4 LDWG Meeting
- **March 2026** Conditional acceptance @ P4 LDWG meeting
- **July 2026** FSE paper
- **October 2026** OOPSLA paper
- **December 2026** Official acceptance @ P4 LDWG meeting (planned)



ESMeta, SpecTec, and P4-SpecTec

	Problems	Requirement	Solution	Burden	Community & Researchers
JS	Bugs in the specification and diverse implementations		Type checking of the specification and rich test generation	ESMeta team's maintenance	
Wasm	Bugs and inconsistencies across artifacts		Automatic generation of the artifacts	Community's learning a specification language	
P4	No specification	P4c	Understanding, discussing, specifying, and testing the semantics	Community's learning a specification language	